

Code Defect Detection with LLM Prompting, LoRA Fine-Tuning, and GraphCodeBERT

Group 8 | Team members: [fill names in alphabetical order by last name]
Course final project report | Code: <https://github.com/JiufuZh/526>

Abstract

This project studies function-level code defect detection as a binary classification task: given a C/C++ function, predict whether it is non-defective or defective. The main research question is whether prompt-only large language models are sufficient for this task, or whether supervised adaptation and code-specific pretraining are necessary. We evaluate a ladder of methods: Qwen2.5-7B-Instruct zero-shot prompting, Qwen 4-shot prompting, Qwen LoRA fine-tuning, majority and TF-IDF CPU baselines, and a GraphCodeBERT encoder classifier. On the held-out test split, GraphCodeBERT achieves the best balanced performance with 0.6589 accuracy and 0.6517 Macro-F1. TF-IDF Logistic Regression is also strong, with 0.6218 Macro-F1 and the highest Defective-F1 of 0.6088. Prompt-only Qwen is weaker, and 4-shot prompting collapses to all non-defective predictions. These results support the conclusion that code-aware supervised models are more reliable than prompt-only LLM use for defect detection.

1. Problem, Motivation, and Novelty

Automated defect detection aims to identify functions that may contain bugs or vulnerabilities before deployment. The motivation is practical: manual code review is costly, defects can be subtle, and false negatives can leave risky code undetected. The project is novel for the course setting because it does not simply fine-tune one model. Instead, it compares a full adaptation ladder from no-training LLM prompting, to in-context prompting, to parameter-efficient LLM fine-tuning, to traditional lexical baselines, to a code-specific transformer encoder. This design lets us analyze not only which model performs best, but what kind of adaptation actually helps.

2. Related Work and Differentiation

The dataset and task follow the CodeXGLUE defect detection benchmark, which is based on the Devign-style function-level defect setting [1, 2]. GraphCodeBERT is a code-pretrained transformer that incorporates code structure signals and is a natural supervised encoder baseline for this task [3]. Qwen2.5-7B-Instruct represents a modern instruction-tuned LLM family that can be evaluated through prompts [4]. LoRA provides a lightweight way to adapt large models without full fine-tuning [5]. Our differentiation is the controlled comparison across these families under the same train/validation/test splits and the same evaluation metrics. The project therefore contrasts prompt-only behavior, parameter-efficient LLM adaptation, lexical supervised baselines, and code-specific supervised pretraining.

3. Dataset and Methods

We use the CodeXGLUE defect detection benchmark. Labels are binary: 0 means non-defective and 1 means defective. The pipeline verifies splits, prepares cached datasets, runs prompt-based LLM inference, trains supervised baselines, evaluates validation and test predictions, and saves metrics plus confusion matrices. The implementation is hosted in the project GitHub repository and the main Tillicum run directory was `/gpfs/projects/imt526a/group8/final`.

Method	Purpose	Adaptation
Majority	Sanity-check baseline	Most frequent class
TF-IDF + SVM/LogReg	Traditional supervised lexical baseline	CPU classifier
Qwen zero-shot	Raw instruction-following LLM reasoning	Prompt only
Qwen 4-shot	In-context examples without training	Prompt examples
Qwen LoRA	Supervised LLM task adaptation	LoRA adapter
GraphCodeBERT	Code-specific supervised encoder	Classifier head training

4. Evaluation Measures

The evaluation uses accuracy, Macro-F1, Defective-F1, defective recall, and confusion matrices. Accuracy is useful but insufficient because a model can obtain moderate accuracy by predicting the majority class while missing all defective functions. Macro-F1 is more appropriate because it gives balanced weight to both classes. Defective-F1 and defective recall are especially important because the high-risk error is failing to catch defective code. The confusion matrix reports true negatives, false positives, false negatives, and true positives so the report can explain how each model behaves.

5. Results

Table 1 summarizes the held-out test results, sorted by Macro-F1.

Model	Acc.	Macro-F1	Def.-F1	Def. recall	TN	FP	FN	TP
GraphCodeBERT	0.6589	0.6517	0.6017	0.5610	1096	381	551	704
TF-IDF Logistic Regression	0.6223	0.6218	0.6088	0.6398	897	580	452	803
TF-IDF Linear SVM	0.6007	0.5994	0.5766	0.5920	898	579	512	743
Qwen LoRA fine-tuned	0.5523	0.5507	0.5236	0.5355	837	640	583	672
Qwen zero-shot	0.5264	0.5180	0.4545	0.4295	899	578	716	539
Qwen 4-shot	0.5406	0.3509	0.0000	0.0000	1477	0	1255	0
Majority baseline	0.5406	0.3509	0.0000	0.0000	1477	0	1255	0

GraphCodeBERT is the strongest balanced model on test, reaching 0.6517 Macro-F1. TF-IDF Logistic Regression is the strongest traditional baseline and is highly competitive, with 0.6088 Defective-F1. Qwen LoRA improves over zero-shot prompting on Macro-F1 and Defective-F1, but it does not reach the supervised lexical or code-specific baselines. The 4-shot model and majority baseline both predict every test sample as non-defective, giving Defective-F1 of 0.

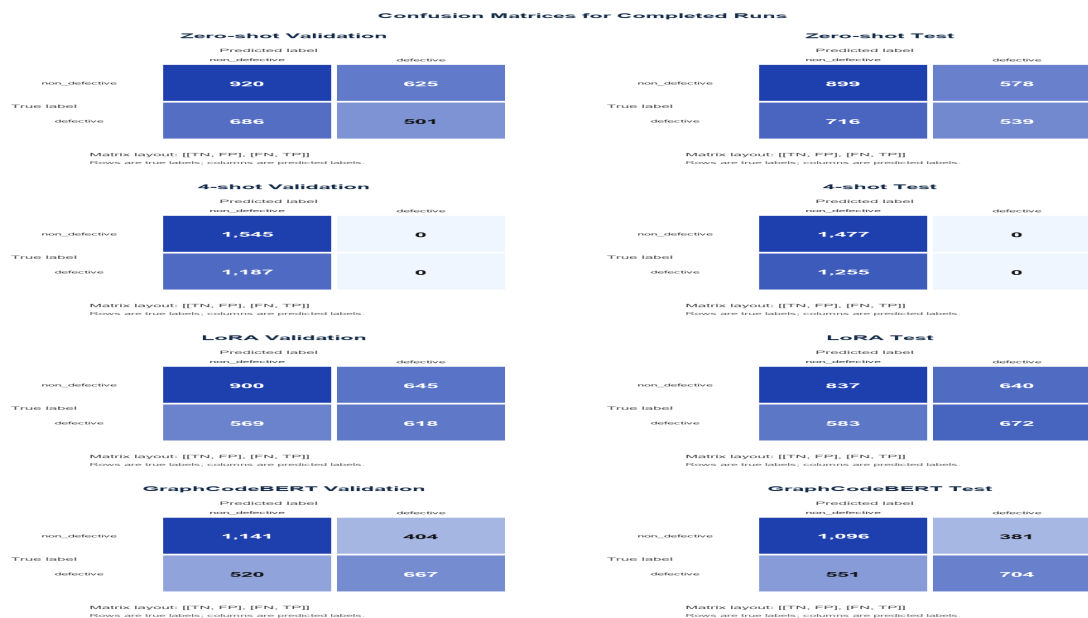


Figure 1. Test/validation confusion matrices show class collapse for 4-shot and majority baselines, and more balanced behavior for supervised models.

6. Analysis of Outcomes

- Prompt-only LLMs are not reliable enough for this task. Zero-shot Qwen reaches 0.5180 test Macro-F1, while 4-shot prompting collapses to non-defective predictions.

- LoRA fine-tuning helps the LLM: test Macro-F1 increases from 0.5180 to 0.5507, and Defective-F1 increases from 0.4545 to 0.5236.
- Traditional supervised baselines are strong. TF-IDF Logistic Regression reaches 0.6218 Macro-F1 and 0.6088 Defective-F1, showing that lexical signals in the function text are meaningful.
- GraphCodeBERT provides the best balanced performance, suggesting that code-specific pretraining is useful when paired with supervised labels.

The error-analysis summaries also suggest that false positives tend to occur on longer and more complex functions than false negatives. On the test split, false positives average 160.4 lines and 15.4 branch points, compared with 52.1 lines and 3.25 branch points for false negatives. This supports the interpretation that complexity and pointer-heavy code can trigger conservative defect predictions.

7. Demo Plan and Reproducibility

The 10-minute demo should use the prepared notebook and repository artifacts rather than retraining live. A professional flow is: introduce the problem and research question, show the method ladder, open the report-ready metrics table, show the confusion matrix figure, briefly show the key scripts/config files, and close with the main takeaway. The demo notebook reads completed outputs, so it is stable for screen sharing. Code, scripts, configs, result JSON files, and generated figures are available at the GitHub link above.

8. Limitations and Future Work

- The best model reaches about 0.65 Macro-F1, so the task is not solved.
- The LoRA run is parameter-efficient but may need threshold tuning, longer context, or more training budget.
- Few-shot prompting may improve with better example selection, but the current naive 4-shot setup is not reliable.
- Future work should test larger code encoders, calibration for defective recall, and deeper error analysis by defect type.

References

- [1] CodeXGLUE benchmark. <https://github.com/microsoft/CodeXGLUE>
- [2] Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. <https://arxiv.org/abs/1909.03496>
- [3] GraphCodeBERT: Pre-training Code Representations with Data Flow. <https://arxiv.org/abs/2009.08366> and <https://huggingface.co/microsoft/graphcodebert-base>
- [4] Qwen model family. <https://huggingface.co/Qwen>
- [5] LoRA: Low-Rank Adaptation of Large Language Models. <https://arxiv.org/abs/2106.09685>
- [6] Project repository and generated code/results. <https://github.com/JiufuZh/526>